

Robot Grid Path Minimization

ALGORITHM DOCUMENTATION & INTERVIEW GUIDE

Step 1: Understand the operation

The robot starts at

(a, b)

We want to reach

$(0,0)$

In one operation we choose a pair

(dx, dy)

$0 \leq dx, dy \leq k$

and move

$(x,y) \rightarrow (x-dx, y-dy)$

So if we choose

$(dx,dy) = (3,5)$

then every time we use this pair we move

Left 3

Down 5

Step 2: Understand the cost

This is the important part.

Suppose we use

$(3,5)$

for the first time.

Cost = 1

Now if later we use

$(3,5)$

again

Cost = 0

Again

$(3,5)$

Cost = 0

Again

$(3,5)$

Cost = 0

So you pay only once per distinct pair.

EXAMPLE

Use

$(2,3)$

$(2,3)$

$(2,3)$

$(2,3)$

Total cost

1

because only one distinct pair exists.

But

$(2,3)$

$(1,4)$

Total cost

2

because two different pairs are introduced.

What are we minimizing?

Not operations.

Not distance.

We minimize

How many different (dx, dy) pairs we need.

So the answer can only be

1

or

2

Why?

Because if one pair cannot do it, two pairs are always enough.
We'll prove that later.

Step 3: When is cost = 1?

Suppose we only use

(dx, dy)

again and again.

Say we use it

m times.

Then total movement becomes

$(m \cdot dx, m \cdot dy)$

We need

$$m \cdot dx = a$$

$$m \cdot dy = b$$

So

$$dx = a/m$$

$$dy = b/m$$

That means

Both

a

and

b

must be divisible by

m

EXAMPLE

$$a=12$$

$$b=18$$

Suppose

$$m=3$$

Then

$$dx=4$$

$$dy=6$$

Use

$$(4,6)$$

three times

$$(12,18) \rightarrow (8,12) \rightarrow (4,6) \rightarrow (0,0)$$

Cost

$$1$$

Exactly the sample.

Step 4: What is the largest possible m?

The largest number dividing both

a and b is

$$\gcd(a,b)$$

Suppose

$$g = \gcd(a,b)$$

Then

$$dx=a/g$$

$$dy=b/g$$

This is the smallest possible step that still keeps the ratio.

EXAMPLE

12 18

$g=6$

$dx=2$

$dy=3$

Repeat

6 times

Works.

Step 5: But k limits dx and dy

We cannot choose

$dx > k$ or $dy > k$

So after dividing by gcd,

we check

$a/g \leq k$ and $b/g \leq k$

If yes

answer = 1.

Otherwise

not possible with one pair.

EXAMPLE 1

$a=12$

$b=18$

$k=8$

gcd

6

$dx=2$

$dy=3$

Both

≤ 8

Works.

Answer

1

EXAMPLE 2

$a=9$

$b=7$

$k=5$

gcd

1

Then

$dx=9$

$dy=7$

Need

$9>5$

Impossible.

Cannot use one pair.

Need answer

2

Why are two pairs always enough?

This is the clever observation.

We can always do

$(a,0)$ repeatedly and $(0,b)$ repeatedly.

Even easier,

because

$$k \geq 1$$

we can always use

$(1,0)$ a times and $(0,1)$ b times.

Distinct pairs used

$(1,0)$

$(0,1)$

Only

2

So answer never exceeds 2.

Therefore

Only two possibilities exist.

1 or 2

The entire intuition

The robot must travel exactly

(a,b)

Suppose we want to pay only once.

Every operation must be identical.

So after repeating one vector

(dx,dy)

multiple times,

the total displacement becomes

$m(dx,dy)$

which must equal

(a,b)

Hence

$(dx,dy) = (a/m, b/m)$

The largest possible

m

is

$\gcd(a,b)$

giving the smallest valid step

$(a/g, b/g)$

If this smallest step already exceeds

k

then every larger step will also exceed k in at least one coordinate, so using a single pair is impossible.

Thus:

If $(a/g, b/g)$ fits within k , answer = 1.

Otherwise, answer = 2.

Interview intuition

If an interviewer asks "Why are we taking the GCD?", you can explain:

"If I want to use only one move vector repeatedly, then the total movement must be an integer multiple of that vector. Therefore, the move vector must divide both a and b . To maximize the number of repetitions (and hence minimize the size of each move so it stays within k), I divide by the greatest common divisor. If even this smallest possible repeated move is larger than k in some coordinate, then no repeated single vector can work, so I need at least two distinct vectors."

That's the key observation the entire problem is built on.

Here is the clean and interview-friendly C++ solution.


```

#include <bits/stdc++.h>
using namespace std;

long long gcd(long long a , long long b){
    if( b == 0)return a;

    return gcd( b , a % b);
}

int main(){
    int t;
    cin>>t;

    while(t--){
        long long x;
        long long y;
        long long k;
        cin>>x>>y>>k;

        int g = gcd(x , y);

        long long dx = x / g;
        long long dy = y / g;
        if( dx <= k && dy <= k){
            cout<<1<<endl;
        }else{
            cout<<2<<endl;
        }
    }

    return 0;
}

```

Time Complexity

For each test case:

$\text{gcd}(a, b) \rightarrow O(\log(\min(a, b)))$

Remaining operations $\rightarrow O(1)$

Overall:

$O(t \cdot \log(\min(a, b)))$

Interview Explanation

If we want the cost to be 1, we must use only one distinct pair (dx, dy).

Suppose we use that pair m times. Then:

$$m * dx = a$$

$$m * dy = b$$

Therefore, m must divide both a and b .

The maximum possible m is $\gcd(a, b)$, giving the smallest repeated move:

$$dx = a / \gcd$$

$$dy = b / \gcd$$

If both dx and dy are at most k , then this single pair can be repeated $\gcd$ times, so the answer is 1.

Otherwise, one pair is impossible. We can always finish using two pairs, such as $(1,0)$ and $(0,1)$, so the answer is 2.

This is the complete logic behind the solution.